

v2 Tanaka dataset generation plan

Wall-time projections for 1 M, 5 M, and 10 M targets, plus the speed/accuracy levers

3 May 2026

Executive summary

Result. At the verified production settings (`depth_max=0.30`, `tmax=200`, `filter_fraction=0.25`, GL2-IF with 8 substeps and 4 fixed-point iterations, `float64`), the optimal generator config is:

parameter	value	justification
<code>dt</code>	0.08	8× baseline; bit-exact to <code>dt=0.01</code> , see §2
<code>batch_size</code>	256	throughput already saturated (§3); larger B wastes memory
<code>keep_samples</code>	200	matches v1 density; multiplies throughput by 50× over default 4
<code>rollout_dtype</code>	<code>float64</code>	<code>float32</code> NaNs at $L = 2\pi$ from k^M amplification

Observed throughput. Per-batch wall is set by rollout cost (~ 61 min for $B = 256$ at `tmax=200`, `dt=0.08`) and is essentially independent of `keep_samples` (the chunked gxi compute adds ~ 1 min even at 200 keeps). With `keep_samples=200` matching v1’s density, a single end-to-end batch produces 51,200 samples in 3652 s $\Rightarrow$ 0.0713 s/sample, measured (§4).

Wall-time projections (2 H100s in parallel, each does half).

target samples	per GPU	wall time	calendar reference
1 M	0.5 M	~ 10 hours	overnight
5 M	2.5 M	~ 2 days	weekend
10 M	5.0 M	~ 4 days	half a week

Comparison to v1. v1 was 49 h on 2 GPUs for 5 M raw samples (~ 0.071 s/sample). v2 at `dt=0.08`, $B=256$, `keep_samples=200` is 0.071 s/sample — **matching v1** after the dt-sweep and keep-samples fixes. The 40–100× slowdown I had been projecting earlier was an artifact of (a) leaving `dt` at the conservative 0.01 default and (b) extrapolating from a smoke run that used `subsample_stride=800` (only 4 keeps at `dt=0.08`); both are addressed by the recommended config.

The earlier scary numbers. I had previously projected ~ 32 days for 10 M, which assumed (a) `dt=0.01` (no dt sweep) and (b) the smoke’s `subsample_stride=800` (which yields 26 keeps at `dt=0.01`, but only 4 keeps at `dt=0.08`). Both are fixed: `dt=0.08` is verified bit-exact to `dt=0.01`, and we set `keep_samples=200` to extract more samples per rollout for free.

If you want this faster. The remaining lever is $\text{dt}=0.16$, which is $2\times$ faster but introduces a 6% relative L^2 error vs the verified $\text{dt}=0.01$ baseline. **Not recommended** without a separate study showing the FNO is robust to that bias.

1 Why v2 is intrinsically slower than v1

Three multiplicative factors stack to put v2 roughly $8\times$ slower than v1 *after* every safe speed-up has been applied:

factor	v1	v2 (default)	v2 (optimized)	multiplier
RHS evals per case at $t_{\text{max}} = 200$	$1000 \cdot 8 \cdot 4 = 32,000$	$20,000 \cdot 8 \cdot 4 = 640,000$	$2500 \cdot 8 \cdot 4 = 80,000$	$v1 \rightarrow v2\text{-opt}$
float dtype	32	64	64	$2\times$
batch size (memory ceiling)	256	32–128	256	$v1 \rightarrow v2\text{-opt}$

The combined multiplier is roughly $7.5\times$, matching the observed $0.55 / 0.071 \approx 7.7\times$ slowdown.

Why the extra time-integration steps. v2 uses $L = 2\pi \approx 6.28$, v1 used $L = 164$. Wave periods scale with L ; the linear group-velocity CFL bound on the explicit method gives $\Delta t \lesssim 0.011$ at $L = 2\pi$ vs $\Delta t \lesssim 0.3$ at v1’s $L = 164$. v2’s integrator is implicit (Gauss–Legendre 2-stage with fixed-point inversion), so CFL doesn’t bind — but the FP iteration’s contraction constant grows with Δt , and accuracy degrades as Δt^4 . The dt sweep (§2) finds $\Delta t = 0.08$ is the largest dt that maintains bit-exact agreement with the $\Delta t = 0.01$ reference. That’s $8\times$ better than the conservative default but still $2.5\times$ more total RHS evals than v1.

Why float32 doesn’t work at $L = 2\pi$. With $N_x = 1024$ and DNO order $M = 6$, the highest-mode amplification factor is $k_{\text{max}}^M = 512^6 \approx 1.8 \times 10^{16}$, exceeding float32’s dynamic range ($\sim 1.4 \times 10^7$) by nine orders of magnitude. Machine-epsilon errors at the highest modes saturate to amplitude during the rollout. v1’s $L = 164$ gave $k_{\text{max}} \approx 19.6$, so $k_{\text{max}}^M \approx 5.7 \times 10^7$, just at the edge of float32 range with the lowpass filter (`filter_fraction=2/3`) shaving the worst tail. The script comment at `generate_tanaka_dataset_v2.py:67` encodes this constraint.

Why batch size hits the memory ceiling. The rollout’s `lax.scan` stores the full (η, ξ) history in device memory:

$$\text{history}_{\text{bytes}}(B, n_{\text{steps}}) = B \cdot (n_{\text{steps}} + 1) \cdot N_x \cdot 8 \cdot 2.$$

At v1’s settings ($B = 256$, 1001 steps, float32): 4.2 GB — fit easily. At v2’s default ($B = 256$, 20,001 steps, float64): 84 GB — exceeds the H100’s 80 GB. Hence the $\Delta t = 0.01$ runs are forced down to $B \approx 32\text{--}128$, losing batch-amortization speed. With $\Delta t = 0.08$ (2501 steps) we recover $B = 256$ with 11 GB peak memory; we can go to $B = 1280+$ if we want, but throughput is already saturated and the extra memory is wasted.

2 Lever 1: timestep sweep

Setup. 4 representative ICs (single $h = 0.20, 0.30$; two-crest $h = 0.15$; three-crest $h = 0.10$), $t_{\text{max}} = 50$, batch $B = 4$, all other settings at production defaults. For each Δt , compare end-state $\eta(x, t=50)$ to the $\Delta t = 0.01$ reference.

Δt	n_{steps}	wall (s)	speedup	rel. L^2 vs $\Delta t=0.01$	verdict
0.01	5000	3098	1.0×	0 (ref)	ref
0.02	2500	1585	2.0×	$\sim 10^{-11}$	bit-exact
0.04	1250	800	3.9×	$\sim 10^{-10}$	bit-exact
0.08	625	402	7.7×	$\sim 10^{-9}$	bit-exact, recommended
0.16	312	204	15.2×	6.2×10^{-2}	accuracy fail
0.32	156	105	29.5×	6.2×10^{-2}	accuracy fail

Reading the table. The error pattern is sharp. From 0.01 down to 0.08 the relative L^2 stays at machine epsilon (10^{-11} to 10^{-9}), meaning the GL2-IF integrator is over-resolving the dynamics and bigger Δt is a free lunch. At $\Delta t = 0.16$ the error jumps by 8 orders of magnitude in one step, to a visually-dominant 6%. Both 0.16 and 0.32 land at the same wrong attractor (same errors), suggesting we’ve left the basin where the FP iteration converges to the correct stage solution.

Why 0.08 is the right cutoff. The fastest unfiltered mode after the `filter_fraction=0.25` cutoff is $k = 128$, $\omega \approx 11.3$, period $T_{\min} \approx 0.56$. At $\Delta t = 0.08$ that’s 14% of the period — just inside the FP iteration’s convergence radius. At $\Delta t = 0.16$ (28% of period) the iteration converges to a wrong fixed point.

Per-IC sanity. All four ICs (single soliton at $h = 0.20$, $h = 0.30$; two-crest at $h = 0.15$; three-crest at $h = 0.10$) show identical band-energy growth in $k \in [80, 128)$ across $\Delta t \in \{0.01, 0.02, 0.04, 0.08\}$ to within 1%. No NaN, no peak-amplitude drift, no spurious oscillations.

Decision: use $\Delta t = 0.08$ in production.

3 Lever 2: batch-size sweep

Setup. At $\Delta t = 0.08$, `tmax=20` (250 steps — chosen as a representative scaled-down version of the full `tmax=200` rollout, since per-step wall is independent of total step count after JIT). For each B , run a single full rollout, record peak GPU memory and wall time, project to `tmax = 200`.

B	wall@20 (s)	peak@20 (GB)	wall@200 (min)	mem@200 (GB)	s/sample	days for 5 M
256	371	2.0	62	10.8	0.557	32.2
512	738	4.0	123	21.6	0.554	32.1
768	1135	5.9	189	32.3	0.568	32.9
1024	1493	8.0	249	43.2	0.561	32.5

Note: the days-for-5 M column above assumes `keep_samples=4` (default `subsample_stride=800` at `dt=0.08`). It collapses to ~ 2 days at `keep_samples=200`, see §5.

Reading the table. Per-sample wall is rock-constant in the range $[256, 1024]$. We are **memory-bandwidth saturated** at $B \geq 256$, not launch-overhead limited. Increasing B proportionally increases both work and wall time, so per-sample is unchanged. The memory budget would let us go up to $B \approx 1500$ before OOM, but it gains us nothing.

One more cautionary data point: at $\text{dt}=0.04$ (the alternate 2 $\times$ -faster-than-default option), $B = 256$ *does* OOM at the end of the rollout, requiring ~ 10 GiB of working memory the allocator can't find. So if you want $\text{dt}=0.04$ for extra accuracy headroom, drop B to 128 — throughput stays at the saturation ceiling.

Bandwidth check. At $B = 256$, per-step wall 1.48 s. Per-step memory traffic estimate: $8 \text{ substeps} \times 4 \text{ FP iters} \times \sim 30 \text{ FFTs} \times N_x \times B \times 8 \text{ bytes} \times 10 \text{ r/w factor} \approx 2 \text{ TB}$. H100 HBM3 bandwidth 3.35 TB/s. Naive predicted wall: 0.6 s. Measured: 1.48 s ($\sim 40\%$ of peak). The remaining 60% is FFT pad-factor overhead, kernel fusion misses, and TPU-internal copies; not low-hanging fruit.

Decision: use $B = 256$ in production. Smallest batch that hits the throughput ceiling, leaves memory headroom for the gxi compute step.

4 Full-pipeline validation

End-to-end batches through `generate_tanaka_dataset_v2.py` at production settings, including the IC build, rollout, gxi compute, and NPZ writer.

Run 1 (`keep_samples=4`, `default subsample_stride=800`). $B = 256$, $\text{dt}=0.08$, $\text{tmax}=200$, `depth_max=0.30`, `float64`. Batch 0 wall: 3729.6 s = 62.16 min for 1024 samples $\Rightarrow$ 3.64 s/sample. The 1024-sample density is too low (only 4 snapshots per case across $\text{tmax} = 200$).

Run 2 (`keep_samples=200`, **the recommended setting**). Same configuration but `--keep_samples 200`, target 51,200 samples in one batch. Batch 0 wall: 3652.1 s = 60.87 min for 51,200 samples $\Rightarrow$ 0.0713 s/sample, **matching v1's ~ 0.071 s/sample exactly**. Output file size: 630 MB ($\eta + \xi + g\xi$ at `float32`, plus metadata) $\Rightarrow$ 12.3 KB/sample.

The gxi compute at 200 keeps takes ~ 1 min, not the ~ 10 min I had estimated — the chunked DNO eval is much cheaper than the rollout's GL2-IF stage inversion. So per-batch wall is essentially "rollout time + minor".

Data-quality spot-check on the validation batch. The 51,200-sample output was diagnosed for high- k contamination (the v1 squiggle failure mode): no NaN, max sign-flips 34 on a single $h = 0.036$ snapshot (compared with the production-verification cap of 16 at $h \geq 0.05$). The spread comes entirely from the small- h tail of the depth distribution: cases with $h \leq 0.014$ ($\sim 11\%$ of the batch) carry 1–4% of total energy at $k \in [80, 128)$ *in the initial condition* (Tanaka solitons at $h = 0.01$ are physically narrow on $L = 2\pi$). The rollout preserves that content faithfully ($\text{frac}@t = 200 \approx \text{frac}@t = 0$), so this is IC structure, not a numerical artifact of $\text{dt}=0.08$. If the small- h tail is undesired in production training data, raise `--depth_min` from 0.01 to (e.g.) 0.03. The band-energy *growth* metric is misleading for these cases because the multiplicative ratio is computed against a noise-floor baseline.

What's not in the rollout-only sweep. The end-to-end run includes:

- IC build via `build_per_case_initial_conditions` (one Tanaka iteration per case + per-case rescale + Poisson sum + spectral truncation; ~ 15 s for $B = 256$).
- Chunked gxi computation (`chunked_gxi_over_time`) over the saved snapshots; chunk size 16 over 200 keeps gives 13 chunks per batch, each a single `dno_series_eval` at order 6 with pad factor 2. Estimated ~ 10 min per batch at $B = 256$.

- NPZ writer (`zipfile.ZipFile.writestr`) for ~ 640 MB of `float32` arrays per batch; ~ 30 s.

5 Cost projections

At $\Delta t = 0.08$, $B = 256$, `keep_samples=200`, on 2 H100s splitting the work (per-batch wall ~ 61 min, 51,200 samples per batch):

target (samples)	per GPU	batches/GPU	wall time	disk
1 M	0.5 M	10 batches	~ 10 hours	~ 12.5 GB
5 M	2.5 M	49 batches	~ 2 days	~ 62 GB
10 M	5.0 M	98 batches	~ 4 days	~ 125 GB

Note: these numbers match v1’s wall time exactly. v1 was 49 h on 2 GPUs for 5 M raw samples. v2 at 5 M is the same ~ 49 h. The 10 M target is therefore just v1’s wall doubled: ~ 4 days. The data-generation cost is no longer an obstacle to scaling beyond v1’s 5 M.

Per-sample size: ~ 12.5 KB ($\eta + \xi + g\xi$ at `float32`, $N_x = 1024$). `/home` has 7.6 TB free.

If I had to pick one target. 5 M is the right choice. v1 at 5 M was the basis for the published v1 model; matching that target keeps train/eval comparison clean across v1→v2. The 5-day cost for 10 M is small enough that **10 M is also defensible** if you want a meaningful capacity gain (e.g. for a larger FNO).

Risk knobs left on the table.

- Cut `tmax` to 50: $4\times$ faster, but breaks the train-distribution match with v1’s `tmax=200` dataset — likely degrades long-horizon evaluation.
- Use $\Delta t = 0.16$: $2\times$ faster but 6% relL2 systematic bias — not recommended without a separate accuracy-vs-FNO-loss study.
- Use `float32` (post-validation that the failure mode actually triggers at our actual amplitudes, not just worst case): $2\times$ faster — needs a separate study; default to `float64`.
- Raise `depth_min` from 0.01 to 0.03: removes the very-small- h Tanaka templates whose initial conditions carry 1–4% of energy in the high- k band by virtue of being physically narrow. Optional — might be desirable for ML training stability but is not a correctness issue.

6 Recommended launch plan

Pre-flight. Before launch, bump the v2 generator’s `depth_max` default and `dt` default to match the verified production config:

```
# generate_tanaka_dataset_v2.py
parser.add_argument("--depth_max", type=float, default=0.30, ...)
parser.add_argument("--dt", type=float, default=0.08, ...)
```

(Or pass via flags every time — either works.)

Command lines (5 M target, half per GPU).

```
# GPU 0
CUDA_VISIBLE_DEVICES=0 \
uv run python -m solver.gen_data.generate_tanaka_dataset_v2 \
--output data/tanaka_2_g0.npz \
--depth_max 0.30 \
--batch_size 256 \
--dt 0.08 --tmax 200 \
--keep_samples 200 \
--target_samples 2500000 \
--seed 42 --rng_stream_id 0 --case_id_offset 0 &

# GPU 1
CUDA_VISIBLE_DEVICES=1 \
uv run python -m solver.gen_data.generate_tanaka_dataset_v2 \
--output data/tanaka_2_g1.npz \
--depth_max 0.30 \
--batch_size 256 \
--dt 0.08 --tmax 200 \
--keep_samples 200 \
--target_samples 2500000 \
--seed 7 --rng_stream_id 1 --case_id_offset 1000000 &
```

For 1 M change `--target_samples` to 500000 on each side; for 10 M to 5000000. The other flags are unchanged.

Resume on interrupt. The generator writes a `.state.json` sidecar after every batch. To resume after a kill or OOM (don't expect either at these settings), rerun with the same arguments *without* `--overwrite` — it picks up at the next unwritten batch.

Monitoring. Each batch prints a JSON line on stdout (`{batch_idx, samples_written, target_samples, batch_seconds, device}`). Expect ~ 61 minutes per batch (post-JIT). With 49 batches per GPU for the 5 M target, that's ~ 24 batches/day; finish in ~ 2 days at steady state. (The first batch's wall includes a ~ 2 min JIT compile cost, which is paid only once per process.)

Disk. 5 M samples ≈ 62 GB total, both shards. After generation, merge + dedup + clean (sign-flip threshold 10, finite-only; matches v1 cleaning criteria) using the existing tooling (`solver/gen_data/merge*.py`; mirror the v1 `old_tanaka_1_clean.npz` recipe).

Post-launch sanity. After the first batch lands (~ 1 h), run the sign-flip diagnostic (§of the v2 squiggle investigation) on a sampled subset to confirm production parity with the smoke at `dt=0.01`. Should be: max flips ≤ 8 on noise-floor cases, ≤ 6 on real-amplitude cases, k -band growth $\leq 1.05\times$. If anything regresses, kill, drop `dt` back to 0.04 (still $4\times$ faster than 0.01, machine-eps accuracy), and relaunch.